

Reproducerea rezultatelor din competiția SAT pentru 2 benchmark-uri non-triviale

Colodi Denis, Pura Fabian, Bobaru Robert, and Ișfan David

Universitatea de Vest din Timișoara
Facultatea de informatică

Abstract. Această lucrare prezintă reproducerea rezultatelor din competiția SAT pentru două benchmark-uri non-triviale. Scopul principal este replicarea și îmbunătățirea performanțelor solverului MiniSAT pe baza unor experimente sistematice. Rezultatele obținute oferă insights cu privire la eficiența algoritmilor SAT și oportunități de optimizare.

1 Introducere

Problema satisfiabilității booleene (SAT) reprezintă una dintre problemele fundamentale din informatica teoretică și practică. Timpul de rezolvare al acestora poate crește exponențial cu dimensiunea problemei. Eficiența solverilor moderni pentru SAT a permis aplicarea acestora în numeroase domenii, cum ar fi verificarea formală a hardware-ului și software-ului, planificare automatizată, bioinformatică și criptografie.

2 Obiective

Scopul acestui proiect este replicarea și îmbunătățirea rezultatelor obținute în cadrul SAT Competition pentru două benchmark-uri non-triviale selectate. În acest proiect, ne propunem să urmărim următoarele obiective:

- Reproducerea independentă a rezultatelor oficiale pentru două benchmark-uri non-triviale selectate din SAT Competition
- Investigarea codului MiniSAT, care include componentele principale și algoritmi utilizați
- Măsurarea diverselor algoritmi a celor două benchmark-uri
- Afișarea rezultatelor sub formă de tabel grafic în urma rulării celor 2 benchmark-uri

În continuare este prezentată arhitectura solverului MiniSAT și rolul componentelor principale în cadrul algoritmului CDCL.

3 Arhitectura MiniSAT

MiniSAT reprezintă un SAT solver minimalist și open-source care este destinat pentru realizarea cercetărilor academice și aplicații practice. O problemă oarecare SAT are ca obiectiv principal să se determine dacă o formulă booleană poate fi interpretată ca fiind adevărată dacă se aleg valori de adevăr pentru variabilele sale. Acest lucru înseamnă că SAT solver-ul interoghează dacă prin asignarea de valori True/False către variabilele unei formule, există posibilitatea ca acea formulă să fie evaluată ca fiind True.

De obicei MiniSAT determină o rezolvare a problemelor de satisfiabilitate a formulelor boolene, cele mai întâlnite fiind cele sub forma conjuncției de clauze CNF, returnând dacă este SAT sau UNSAT [1].

MiniSAT implementează un algoritm de tip DPLL care este extins mai departe cu CDCL, acest lucru înseamnă că se pornește de la formula CNF, iar ulterior se alege o variabilă recursivă neasignată căreia i se atribuie valoarea de adevăr TRUE sau FALSE. După fiecare decizie, are loc realizarea propagării unitare unde o variabilă este forțată la valoarea de care este nevoie pentru a determina clauza TRUE în cazul în care există clauze care conțin o singură variabilă neasignată, iar celelalte sunt False, procesul repetându-se până când nu mai există alte clauze unitare. Atunci când există conflicte, se face o analiză asupra acestuia, urmând lanțul de propagări pentru a genera o clauză conflict care împiedică aceeași combinație de decizii. Următorul pas este backjumping-ul, care presupune revenirea la un nivel de decizie mult mai vechi ce este determinat de analiza conflictului și de clauza învățată. Atunci când la nivelul 0 este prezent conflictul, acest proces se va opri, rezultând că formula este UNSAT [2].

Din punct de vedere al algoritmilor și a tehnicilor de implementare, minisat combină o serie de tehnici mai avansate din arhitectura CDCL care ajută la sporirea eficienței. Printre aceste tehnici se pot enumera propagarea unitară cu doi literal, învățarea de clauze, backjumping-ul necronologic sau gestionarea memoriei clauzelor. Propagarea unitară presupune că pentru fiecare clauză, MiniSAT urmărește 2 literale neasignate. Astfel s-a dedus că această metodă este mult mai eficientă deoarece un literal monitorizat care devine False este verificat de clauză dacă trebuie propagat [2].

De fiecare dată când apar conflicte, MiniSAT le analizează cu ajutorul metodei Marques-Silva și Sakallah ce presupune generarea de clauze împotriva conflictelor. Aceste clauze învățate sunt ulterior adăugate în baza de date și nu pot fi satisfăcute de modelul curent [2].

Backjumping-ul, așa cum este amintit mai devreme, nu reia nivelul de decizie de la pasul imediat anterior, ci la nivelul de decizie care corespunde cu cele mai scăzute presupuneri din clauza pe care a învățat-o, trecând peste deciziile care nu au nicio influență, fiind astfel o abordare foarte eficientă [2].

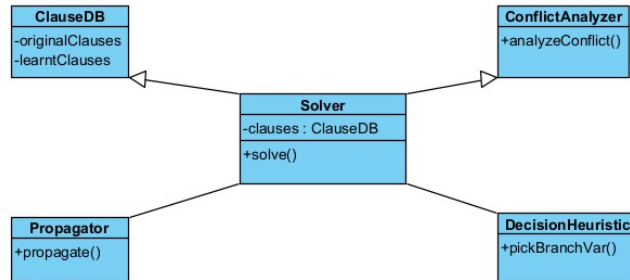


Fig. 1: Arhitectura solverului SAT bazat pe algoritmul CDCL.

Diagrama din Figura 1 ilustrează structura internă a solverului SAT și interacțiunea dintre principalele sale componente.

Componenta centrală este **Solver**, care coordonează execuția globală a algoritmului CDCL. Aceasta deține o referință către **ClauseDB**, baza de clauze ce include atât clauzele CNF originale, cât și clauzele învățate pe parcursul rezolvării. Metoda `solve()` implementează bucla principală a algoritmului, gestionând selecția variabilelor, propagarea constrângerilor, detectarea conflictelor și decizia de satisfiabilitate.

ClauseDB stochează formulele logice utilizate în procesul de rezolvare, fiind accesată de Solver în timpul propagării și extinsă cu noi clauze rezultate din analiza conflictelor.

Deducția logică este realizată de componenta **Propagator**, prin metoda `propagate()`, care implementează mecanismul de Boolean Constraint Propagation (BCP). La fiecare atribuire, Propagator actualizează implicațiile logice, deduce valori noi pentru variabilele unitare și semnalează eventualele conflicte.

În cazul apariției unui conflict, **Conflict Analyzer** este apelat pentru a identifica cauza acestuia. Prin metoda `analyzeConflict()`, se generează o clauză învățată și se determină nivelul de backjump necesar pentru continuarea eficientă a căutării.

Alegerea variabilelor de decizie este realizată de **DecisionHeuristic**, prin metoda `pickBranchVar()`, care selectează variabila neasignată cu cel mai mare scor de activitate. Decizia este aplicată de Solver pentru a explora în continuare spațiul de căutare.

4 Benchmark-uri Selectate

4.1 Two-trees benchmark

Benchmark-ul Two-Trees reprezintă un set de date de testare artificial cu un grad ridicat de dificultate în domeniul rezolvării problemelor SAT. Acesta conține 511 variabile și 2039 clauze, valori care îl plasează în categoria problemelor de dimensiune medie, dar concepute deliberat pentru a provoca mecanismele euristice ale solverelor. Denumirea two-trees sugerează o structură logică derivată din doi arbori interconectați, ceea ce generează o topologie neuniformă și crește complexitatea decizională în procesul de căutare.

În plus, acest benchmark servește ca instrument de referință pentru a demonstra beneficiile tehnicilor avansate introduse în solvele moderne, cum ar fi: învățarea clauzelor (permite solverului să evite refacerea acelorași greșeli prin deducerea unor clauze noi rezultante din conflicte), backjumping-ul (permite revenirea la un nivel de decizie mai potrivit în arborele de căutare, în locul unei întoarceri pas cu pas) și euristici de selectare a variabilelor (pentru ghidarea eficientă a procesului de căutare).

Pe măsură ce SAT solvele devin tot mai sofisticate, la fel și benchmark-urile trebuie dezvoltate pentru a reflecta provocări relevante și actuale. Problemele de testare precum Two-Trees sunt importante în evaluarea performanței algoritmilor, în comparația între tehnici diferite și în direcționarea progresului din domeniul satisfiabilității booleene.

În final, o problemă SAT poate fi satisfiabilă (există o atribuire de valori TRUE/FALSE care face formula adevărată) sau nesatisfiabilă (UNSAT), caz în care nu există nicio atribuire posibilă care să satisfacă formula. Distincția este fundamentală în analiza performanței solverelor și în dezvoltarea unei teorii robuste privind complexitatea instanțelor SAT [2].

4.2 X9-09054 benchmark

Acest benchmark derivă dintr-o problemă practică din domeniul industrial. Denumirea sa indică faptul că face parte dintr-un set de instanțe specifice domeniului Electronic Design Automation (EDA), mai exact din design-ul circuitelor logice configurabile, cum ar fi FPGA-urile (Field-Programmable Gate Arrays). Formularea CNF a benchmark-ului corespunde unei traduceri logice a unei sarcini reale de rutare sau de mapare logică într-un FPGA.

Spre deosebire de benchmark-uri artificiale precum „two-trees”, care sunt construite ad hoc pentru a explora capacitățile euristice ale solverelor SAT, acest set de date reflectă o aplicație industrială autentică. În contextul EDA, solvele SAT sunt importante pentru îndeplinirea unor numeroase sarcini critice, precum: verificare formală, verificarea echivalenței logice dar și în optimizarea rutării FPGA.

Rezolvarea problemelor SAT provenite din EDA este foarte complexă, deoarece instanțele industriale pot implica un număr mult mai mare de clauze și o structură mult mai complexă decât cele artificiale. Structura clauzelor nu este generată aleator, ci reflectă constrângeri reale impuse de topologia circuitelor, de

limitările de rutare și de regulile de arhitectură FPGA. Aceasta testează nu doar viteza brută a solverului, ci și capacitatea lui de a exploata informația structurală derivată din domeniul hardware.

Companii mari de EDA, precum Cadence și Synopsys, utilizează aceste tipuri de benchmark-uri industriale pentru validarea și testarea solverelor SAT integrate în uneltele lor comerciale. Prin aceasta, ele se asigură că solvelele folosite intern pot face față problemelor pe care le ridică clienții reali: instanțe scalabile, cu decizii critice și cu topologii complexe [3].

5 Analiza performanței solverului MiniSat pentru cele două instanțe SAT

În cadrul proiectului au fost analizate două fișiere CNF – `x9-09054.sat.sanitized.cnf` și `two-trees-511v.sanitized.cnf` – pentru a observa modul în care solverul MiniSat se comportă pe instanțe diferite ca structură și dificultate. Compararea se bazează pe indicatori utilizați în analiza solverelor SAT: conflicte, decizii, propagări, timp de execuție și memorie utilizată. Valorile sunt prezentate în Tabelul 1.

Table 1: Compararea parametrilor de performanță pentru cele două instanțe SAT

Parametru	<code>x9-09054.cnf</code>	<code>two-trees-511v.cnf</code>
Număr variabile	450	511
Număr clauze	4052	2039
Timp execuție	238 sec (~ 4 min)	1320 sec (~ 22 min)
Conflicte	~5.0 milioane	~42.5 milioane
Decizii	~6.2 milioane	~46 milioane
Propagări	~485 milioane	~4.1 miliarde
Memorie utilizată	~57 MB	~78 MB

5.1 `x9-09054.sat.sanitized.cnf`

Această instanță conține 450 variabile și 4052 clauze. Rezolvarea a fost finalizată în aproximativ 238 secunde (aproape 4 minute). În acest interval, solverul a înregistrat în jur de 5 milioane conflicte, peste 6.2 milioane decizii, aproximativ 485 milioane propagări și un consum de memorie de aproximativ 57 MB.

Valorile obținute arată că instanța necesită un volum de lucru semnificativ, însă solverul reușește să parcurgă formula într-un timp relativ scurt, ceea ce sugerează o structură care permite avansarea eficientă în căutare.

5.2 `two-trees-511v.sanitized.cnf`

Această instanță cuprinde 511 variabile și 2039 clauze. Solverul a necesitat aproximativ 1320 secunde (aproximativ 22 minute) pentru a o rezolva. Pe parcurs au fost raportate peste 42.5 milioane conflicte, peste 46 milioane decizii, aproximativ 4.1 miliarde propagări și un consum de memorie de aproximativ 78 MB.

Timpul de execuție mult mai mare și numărul foarte ridicat de operații indică faptul că această formulă este considerabil mai dificil de procesat. Deși are mai puține clauze decât prima, modul în care sunt structurate acestea determină un proces de căutare mult mai intens.

5.3 Comparatie între cele două benchmark-uri

Compararea celor două benchmark-uri evidențiază faptul că dificultatea unei instanțe SAT nu depinde doar de dimensiunea ei numerică (numărul variabilelor sau al clauzelor), ci în mare parte de organizarea internă a formulei. Prima instanță (**x9-09054**) este rezolvată într-un timp relativ redus, în ciuda numărului mare de clauze (Fig 2). A doua instanță (**two-trees-511v**) necesită însă un efort mult mai mare din partea solverului, lucru reflectat de creșterea conflictelor, propagărilor și timpului total de rezolvare (Fig 3).

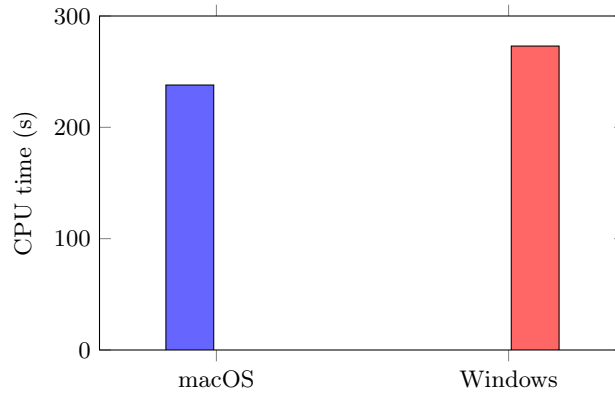


Fig. 2: Timpul de procesare pentru benchmark-ul x9-09054

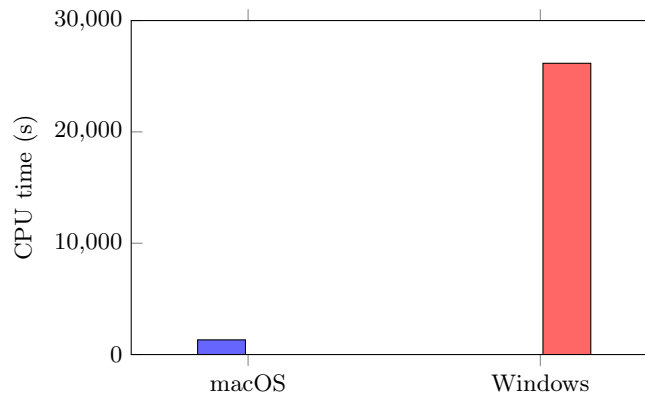


Fig. 3: Timpul de procesare pentru benchmark-ul two-trees-511v

6 Provocări și Limitări

Principala dificultate întâmpinată în timpul desfășurării proiectului a fost timpul de execuție excesiv al solverului SAT. Mai multe instanțe de problemă au necesitat perioade îndelungate de rulare, fiind observați timpi de rulare de până la șapte ore pentru Windows.

În plus, un set al benchmark-urilor nu a finalizat execuția nici după un interval continuu de 24 de ore. În consecință, au fost necesare compromisuri între acoperirea benchmark-urilor și fezabilitatea computațională, fiind ales un set de benchmark-uri care au un timp de rulare rezonabil.

7 Concluzii

În cadrul acestei lucrări a fost analizat comportamentul solverului MiniSAT prin reproducerea și evaluarea performanței sale pe două benchmark-uri non-triviale, provenite atât din instanțe generate artificial în scopul evaluării performanței solverelor SAT, cât și din aplicații industriale. Studiul a urmărit înțelegerea arhitecturii interne a solverului, a mecanismelor fundamentale ale algoritmului CDCL și a modului în care acestea influențează eficiența procesului de rezolvare.

Rezultatele obținute evidențiază faptul că performanța unui solver SAT nu este determinată exclusiv de dimensiunea numerică a instanței (numărul de variabile sau clauze), ci într-o mare măsură de structura internă a formulei. Deși benchmark-ul two-trees-511v conține mai puține clauze decât instanța x9-09054, complexitatea sa structurală conduce la un număr mult mai mare de conflicte, propagări și un timp de execuție semnificativ mai ridicat.

În concluzie, lucrarea demonstrează importanța alegerii benchmark-urilor potrivite pentru evaluarea solverelor SAT și confirmă eficiența arhitecturii MiniSAT în rezolvarea instanțelor de dificultate variată.

References

1. *Boolean satisfiability problem*. https://en.wikipedia.org/wiki/Boolean_satisfiability_problem
2. N. Eén and N. Sörensson, “An extensible SAT-solver,” in Proceedings of the International Conference on Theory and Applications of Satisfiability Testing (SAT 2003), 2003.
3. S. Pakin, “Two Trees: Producing Hard, Satisfiable SAT Benchmarks with Planted Solutions,” Computer, Computational, and Statistical Sciences Division, Los Alamos National Laboratory, Los Alamos, New Mexico, USA 87545.
4. Knight Tours Forum, “Hard Randomized Small HCP Benchmarks,” <https://knighttours.createaforum.com/general-hamiltonian-cycles-problem/hard-randomized-small-hcp-benchmarks/>,